

Ex.no:9	MINI PROJECT – SOCIAL MEDIA ANALYTICS
	DATA ANALYTICS

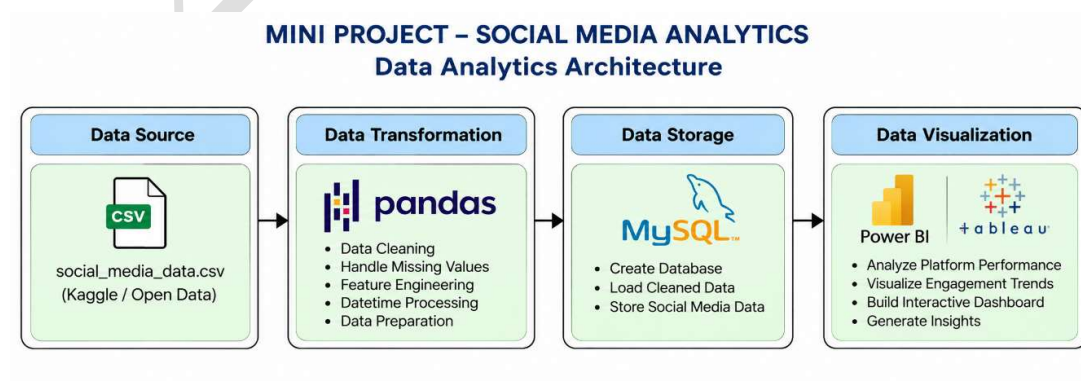
Aim:

To clean, transform, store, and analyze social media data using Python, MySQL, Power BI and Tableau to derive meaningful insights into platform performance, post types, engagement, reach, and posting trends.

Algorithm:

1. Load the social media dataset from the CSV file using Pandas.
2. Inspect the dataset for missing values and duplicate records.
3. Handle missing categorical and numerical values using suitable imputation methods.
4. Convert the Timestamp field into datetime format and reconstruct any missing timestamp using the available date and time fields.
5. Remove duplicate records and standardize the column names.
6. Create derived attributes such as total_engagement, engagement_per_reach, and day_of_week.
7. Store the cleaned and transformed dataset in a MySQL database named social_media_analytics.
8. Connect the MySQL database to Power BI for analysis and reporting.
9. Create DAX measures for total posts, likes, comments, shares, total engagement, average engagement rate, and average reach.
10. Develop Power BI and Tableau visualizations to analyze platform, post type, category, day, hour, reach, impressions, and engagement.
11. Generate meaningful insights from the dashboards.
12. End the process.

Architecture:



Program:

1. Python – Data Cleaning & Transformation

```
import pandas as pd

# Load dataset
df = pd.read_csv("social media dataset.csv", sep=";")

# Create a copy for processing
social = df.copy()

# Check missing values and duplicate records
print("Original Shape:", social.shape)
print("\nMissing Values Before Cleaning:")
print(social.isnull().sum())
print("\nDuplicate Records Before Cleaning:",
      social.duplicated().sum())

# Remove duplicate records
social.drop_duplicates(inplace=True)

# Handle missing categorical values
categorical_columns = [
    "Area", "Category", "Platform",
    "Post Type", "User"
]
for column in categorical_columns:
    social[column] = social[column].fillna("Unknown")

# Convert numerical columns
numeric_columns = [
    "Day", "Month", "Week", "Year",
    "Comments", "Engagement Rate",
    "Hour", "Impressions", "Likes",
    "Reach", "Shares"
]
for column in numeric_columns:
    social[column] = pd.to_numeric(
        social[column].astype(str).str.replace(",", ".", regex=False),
        errors="coerce"
    )
    social[column] = social[column].fillna(
        social[column].median()
    )

# Convert Timestamp to datetime
social["Timestamp"] = pd.to_datetime(
    social["Timestamp"],
    format="%d/%m/%Y %H:%M:%S",
    errors="coerce"
)

# Fix missing timestamp
missing_timestamp = social["Timestamp"].isna()
social.loc[missing_timestamp, "Timestamp"] = pd.to_datetime(
    social.loc[missing_timestamp, "Year"].astype(int).astype(str)
    + "-" + social.loc[missing_timestamp, "Month"].astype(int).astype(str)
    + "-" + social.loc[missing_timestamp, "Day"].astype(int).astype(str)
    + " " + social.loc[missing_timestamp, "Hour"].astype(int).astype(str)
    + ":00:00"
)

# Create derived columns
social["Total Engagement"] = (
    social["Likes"] + social["Comments"] + social["Shares"]
)
```

```

    )

    social["Engagement per Reach"] = (
        social["Total Engagement"] / social["Reach"]
    ) * 100

    social["Day of Week"] = (
        social["Timestamp"].dt.day_name()
    )

    # Standardize column names
    social.columns = (
        social.columns.str.lower()
        .str.strip()
        .str.replace(" ", "_")
    )

    print("\nMissing Values After Cleaning:")
    print(social.isnull().sum())
    print("\nDuplicate Records After Cleaning:",
          social.duplicated().sum())
    print("\nFinal Shape:", social.shape)
    print("\nData Types After Cleaning:")
    print(social.dtypes)
    print("\nFirst 5 Cleaned Records:")
    display(social.head())

```

Python – Output:

Original dataset shape: 733 rows × 17 columns.

Cleaned dataset shape: 733 rows × 20 columns.

The cleaned dataset contains the original attributes together with total_engagement, engagement_per_reach and day_of_week.

Original Shape: (733, 17)

Missing Values Before Cleaning:

Area	0
Category	1
Day	1
Month	1
Platform	1
Post Type	1
Timestamp	1
User	1
Week	1
Year	1
Comments	1
Engagement Rate	1
Hour	1
Impressions	1
Likes	1
Reach	1
Shares	1
dtype: int64	

Duplicate Records Before Cleaning: 0

Missing Values After Cleaning:

area	0
category	0
day	0
month	0
platform	0
post_type	0
timestamp	0
user	0
week	0
year	0

Final Shape: (733, 20)

Data Types After Cleaning:

area	str
category	str
day	float64
month	float64
platform	str
post_type	str
timestamp	datetime64[us]
user	str
week	float64
year	float64
comments	float64
engagement_rate	float64
hour	float64
impressions	float64
likes	float64
reach	float64
shares	float64
total_engagement	float64
engagement_per_reach	float64
day_of_week	str
dtype: object	

comments	0
engagement_rate	0
hour	0
impressions	0
likes	0
reach	0
shares	0
total_engagement	0
engagement_per_reach	0
day_of_week	0
dtype: int64	

Duplicate Records After Cleaning: 0

First 5 Cleaned Records:

	area	category	day	month	platform	post_type	timestamp	user	week	year	comments	engagement_rate	hour	impressions	likes	reach	sha
0	Aceh	Medium	15.0	1.0	Twitter	Video	2023-01-15 12:30:00	User123	3.0	2023.0	77.0	31.32	12.0	2619.0	415.0	1842.0	8
1	Bali	Low	15.0	1.0	Twitter	Video	2023-01-15 08:45:00	CommuterX	3.0	2023.0	125.0	28.78	8.0	1223.0	908.0	3884.0	8
2	Bangka Belitung	High	15.0	1.0	Instagram	Video	2023-01-15 15:45:00	FitnessFan	3.0	2023.0	419.0	104.71	15.0	2165.0	412.0	849.0	5
3	Banten	Low	15.0	1.0	TikTok	Image	2023-01-15 18:20:00	AdventureX	3.0	2023.0	153.0	24.13	18.0	7191.0	667.0	3519.0	2
4	Bengkulu	Medium	15.0	1.0	Instagram	Image	2023-01-15 19:55:00	ChefCook	3.0	2023.0	215.0	44.69	19.0	3312.0	659.0	2032.0	3

2. MySQL – Database and Data Loading

MySQL:

```
CREATE DATABASE IF NOT EXISTS social_media_analytics;
```

```
USE social_media_analytics;
```

Python/Jupyter:

```
from sqlalchemy import create_engine
```

```
username = "root"
```

```
password = "YOUR_MYSQL_PASSWORD"
```

```
host = "localhost"
```

```
port = 3306
```

```
engine = create_engine(
    f'mysql+pymysql://{username}:{password}'
    f'@{host}:{port}/social_media_analytics'
)
```

```
social.to_sql(
    name="social_media_posts",
    con=engine,
    if_exists="replace",
    index=False
)
```

```
print("Cleaned data loaded into MySQL successfully.")
```

```
print("Number of rows loaded:", len(social))
```

3. MySQL – Analysis Queries

1. Total records

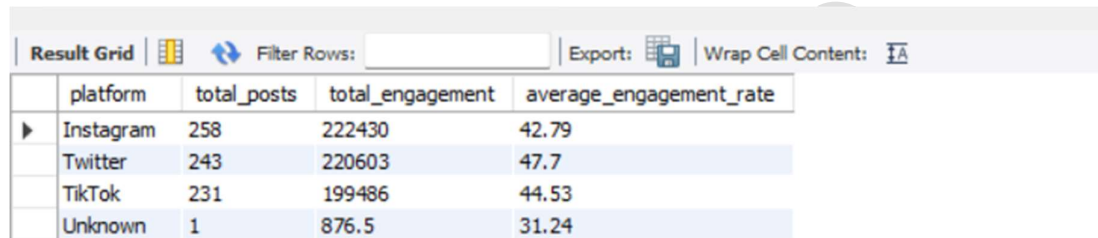
```
SELECT COUNT(*) AS total_records
```

```
FROM social_media_posts;
```

Result Grid		Filter Rows:	Export:	Wrap Cell Content:
	total_records			
	733			

2. Platform-wise performance

```
SELECT
    platform,
    COUNT(*) AS total_posts,
    SUM(total_engagement) AS total_engagement,
    ROUND(AVG(engagement_rate), 2)
    AS average_engagement_rate
FROM social_media_posts
GROUP BY platform
ORDER BY total_engagement DESC;
```

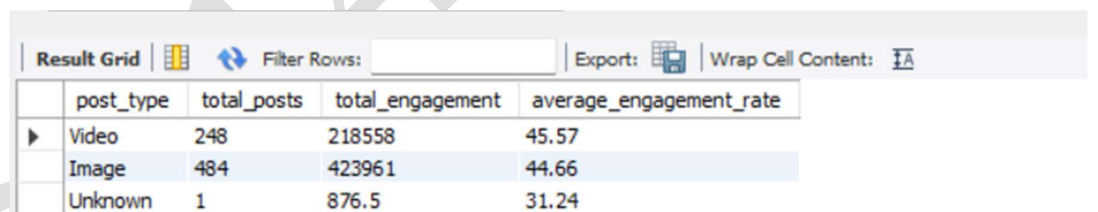


The screenshot shows a database query result grid with the following data:

	platform	total_posts	total_engagement	average_engagement_rate
▶	Instagram	258	222430	42.79
	Twitter	243	220603	47.7
	TikTok	231	199486	44.53
	Unknown	1	876.5	31.24

3. Post type-wise performance

```
SELECT
    post_type,
    COUNT(*) AS total_posts,
    SUM(total_engagement) AS total_engagement,
    ROUND(AVG(engagement_rate), 2)
    AS average_engagement_rate
FROM social_media_posts
GROUP BY post_type
ORDER BY average_engagement_rate DESC;
```



The screenshot shows a database query result grid with the following data:

	post_type	total_posts	total_engagement	average_engagement_rate
▶	Video	248	218558	45.57
	Image	484	423961	44.66
	Unknown	1	876.5	31.24

4. Day-wise engagement

```
SELECT
    day_of_week,
    COUNT(*) AS total_posts,
    SUM(total_engagement) AS total_engagement,
    ROUND(AVG(engagement_rate), 2)
    AS average_engagement_rate
FROM social_media_posts
GROUP BY day_of_week
ORDER BY average_engagement_rate DESC;
```

	day_of_week	total_posts	total_engagement	average_engagement_rate
►	Monday	97	87118	51.72
	Saturday	115	107818	49.47
	Tuesday	111	97958.5	48.8
	Wednesday	88	82810	48.37
	Friday	108	89686	40.99
	Thursday	95	78410	38.92
	Sunday	119	99595	37.35

3. Power BI – DAX Measures

Total Posts

Total Posts = COUNTROWS('social_media_analytics social_media_posts')

Total Likes

Total Likes = SUM('social_media_analytics social_media_posts'[likes])

Total Comments

Total Comments = SUM('social_media_analytics social_media_posts'[comments])

Total Shares

Total Shares = SUM('social_media_analytics social_media_posts'[shares])

Total Engagement

Total Engagement = SUM('social_media_analytics
social_media_posts'[total_engagement])

Average Engagement Rate

Average Engagement Rate = AVERAGE('social_media_analytics
social_media_posts'[engagement_rate])

Average Reach

Average Reach = AVERAGE('social_media_analytics social_media_posts'[reach])

Power BI visuals created include KPI cards, total engagement by platform, total engagement by post type, total posts by category, total engagement by day of week, total engagement by hour, and reach versus impressions.

Tableau Dashboard

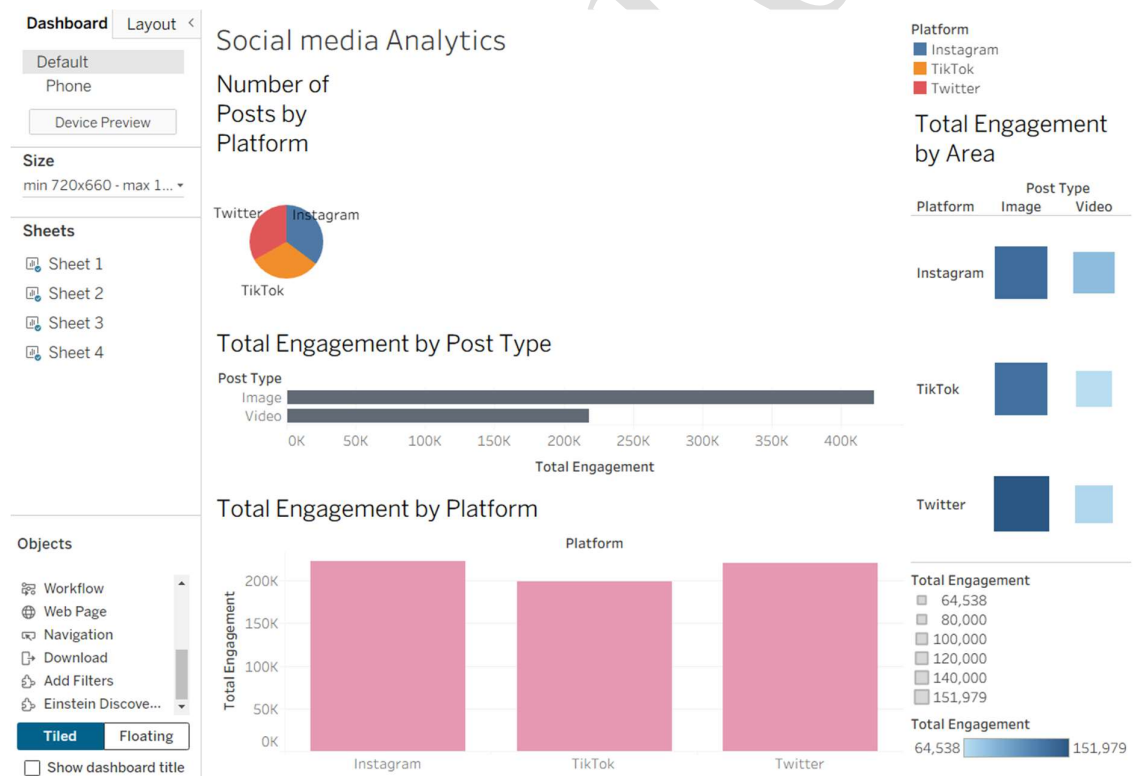
The cleaned CSV file was connected to Tableau Public and separate worksheets were created for social media analysis.

Tableau worksheets used:

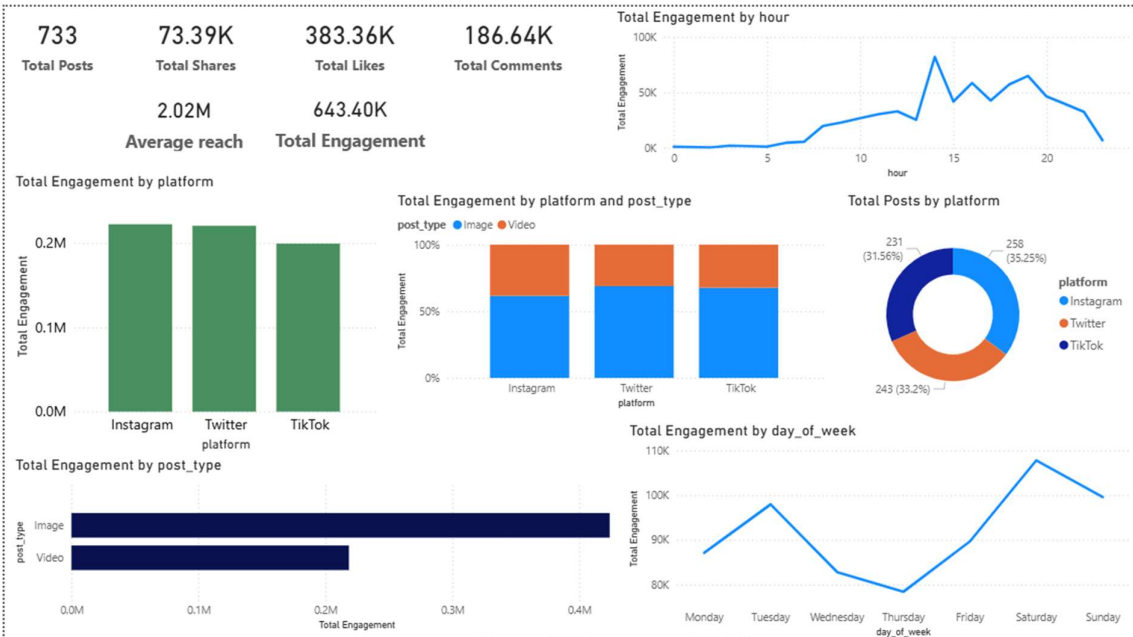
1. Engagement by Platform
2. Engagement by Post Type
3. Number of Posts by Platform
4. Platform × Post Type Heatmap

Output:

TABLEAU DASHBOARD



POWER BI DASHBOARD



Result: